The Red Queen Runs on LLMs: Taxonomy, Evaluation, and Co-Evolution of LLM-Driven Cybersecurity

Aditya Singh

***Abstract*—Large language models stopped answering security questions and started doing security work. We track that shift across 36 records (academic papers, competition artifacts, industry disclosures, and registry entries) plotted on two axes: who chooses the next action (A0 through A3, from prompted assistant to autonomous production hunter), and where in the offense/defense loop the system operates (B1 through B4). Fourteen records received autonomy assessment. Pipeline automation (A1) already runs at ecosystem scale. Task agents (A2) show up in seeded production research, entry-point-assisted auditing, black-box platform operation, and DARPA's AI Cyber Challenge scored rounds. Nothing in the corpus meets our A3 definition. We also trace how offense and defense feed each other, documenting one clean cross-program link (Project Zero's SQLite campaign citing AIxCC as inspiration) while refusing to generalize to field-wide co-evolution. Evaluation validity turns out to be a measurable constraint: information conditions swing reported exploitation rates by more than an order of magnitude, dataset reconstruction collapses detection scores, and proof-of-vulnerability oracles let teams game the metric without fixing the bug. We end with a roadmap held to its own falsifiers, from autonomy-versus-scale ablations to decision-impact audits aimed at making an increasingly autonomous practice also a more verifiable one.**

1. Introduction

Something changed between 2024 and 2026. Vulnerability discovery, exploitation, and patching acquired a new kind of operator: software systems where large language models choose and execute security-relevant actions. Google's agent found an exploitable memory-safety zero-day in SQLite that 150 CPU-hours of AFL could not rediscover [1]. A commercial agent briefly held the top US position on a major bug-bounty platform [2]. A government challenge scored seven fully autonomous systems on finding, proving, and patching vulnerabilities in open-source software [3], [4]. Anthropic reported, as a single-source threat-intelligence assessment, an intrusion campaign whose execution was 80–90% autonomous [5]. These events share no benchmark and no metric. That is exactly why existing surveys cannot answer the questions they raise.

1.1. The Empirical Problem

Three problems block straightforward assessment. First, *unit confusion*: results are reported about models (benchmark pass rates), pipelines (fuzz-target generation), agents (tool-using loops), and organizations (leaderboard ranks) as if they measure the same thing. They do not. Second, *loop fragmentation*: discovery, validation, and repair get evaluated separately even when deployed systems integrate them. Third, *validity opacity*: headline numbers travel without the information conditions, label provenance, or adjudication rules that produced them. GPT-4-based agents exploited 87% of one-day CVEs when given their descriptions and 7% when not [6], yet the qualified number is the one usually quoted.

1.2. Our Approach

We build a PRISMA-informed evidence base from three streams (academic literature, competition and industry documents; registry records) and score every record on two axes. The first is a *decision-authority* gradient A0–A3, assigned strictly by documented behavior: who chooses the next action? Autonomy claims get graded by their weakest missing condition. The second is the offense/defense loop positions B1–B4, recorded as sets for systems that span phases. Thirteen in-window records receive definitive autonomy classifications; one remains unresolved; the remaining 22 provide context, validity evidence, loop-transition documentation, or positioning against prior work. We keep the distinction explicit throughout.

1.3. Contributions

C1. An autonomy×loop taxonomy applied record-by-record with published behavioral evidence (Secs. 4, 5; Fig. 2). **C2.** An evidence-graded analysis of offense↔defense coupling, including a documented causal edge from AIxCC to Project Zero's research program (Sec. 8). **C3.** A systematization of evaluation-validity failures as measurable phenomena, with a five-part contamination-control protocol (Sec. 9). **C4.** A falsifier-specified experimental roadmap (Sec. 11).

1.4. Insights

I1. Evaluation, deployment, and discourse increasingly measure progress by agentic autonomy rather than model

capability alone; whether this reflects a real shift in underlying capability remains unresolved. **I2.** Multiple documented offense↔defense coupling edges exist, including one explicit cross-program inspiration, but the evidence does not establish field-wide causal co-evolution. **I3.** Evaluation validity is a binding methodological constraint on what this field can claim to know, pending the decision-impact audit (E7) that would confirm or refute this.

1.5. Scope

We cover January 2024–June 2026, with pre-2024 work as background only. We classify *documented behavior*, not marketing; we distinguish vendor-reported from independently verified results everywhere. No included evidence met our operational definition of A3. We include Agentless [7] only as a security-relevant repair baseline where its fixed three-stage architecture (localization, repair, and validation) is directly compared with agentic vulnerability repair systems; it is classified as A1 because its workflow follows a predetermined sequence rather than letting the model choose what to do next.

2. Background and Related Work

2.1. Background

Three technical threads meet in this paper. *Code LLMs* provide the reasoning substrate; their security-relevant evaluations started with secure-coding benchmarks [8] and matured into risk suites probing cyberattack helpfulness [9], [10]. *Vulnerability detection* supplies B1's evaluation layer, where PrimeVul demonstrated that label noise and temporal leakage had inflated a generation of results [11], a critique Risse and Böhme systematized from measurement theory [12]. Ullah et al. [13] found early on that LLMs cannot reliably identify and reason about security vulnerabilities, the baseline skepticism our validity section extends. Li et al. [14] show that hybrid approaches combining LLM-assisted static analysis with traditional pattern matching outperform either component alone on real-world datasets. *Interactive agent harnesses* descend from execution-feedback benchmarks like InterCode-CTF (pre-2024 background) [15] through NYU CTF Bench [16] and Cybench [17]. PentestGPT established the penetration-testing tree (PTT) as a planning data structure for LLM-empowered automated penetration testing, showing that domain-grounded agent architectures beat raw prompting on real-world targets [18]. The Model Context Protocol ecosystem, measured at scale for server-side insecurity [19] and analyzed for tool-poisoning, spec-level, and governance risks [20]–[22], supplies the tool-integration substrate on which A2 autonomy operates.

2.2. Systematizations We Position Against

Table 1 codes the seven closest prior efforts. R1's SLR mapped 300+ LLM×security works but closed before the agentic turn [23]; R2 covers detection-to-remediation with no exploitation axis [24]; R3, the closest survey chronologically, organizes pentesting only, treats "co-evolution" as an architecture–benchmark relationship internal to that subfield, and excludes competition-grade systems by corpus design; it also falls outside our window (July 2026), so we treat it as a comparator, not migration evidence [25]. R4 is detection-centric within software security [26]. R5 systematizes the security of agent systems (agents as attack surface) whereas our subject is agents as operators of the vulnerability loop [27]. R6's TOSEM SLR contributes 263 detection studies through November 2025 but carries no offense axis and no industrial track [28]. R7, the AIxCC systematization, is our most important predecessor: we cite-and-exceed it by placing CRSs on an autonomy gradient, extending analysis to post-competition redeployment (B4), and adding the validity critique its design-focused treatment does not attempt [4]. Chen et al. [29] independently synthesize 39 frontier cybersecurity benchmarks across task complexity and domain-specific evaluation settings, finding a systemic gap between static benchmark success and end-to-end adversarial efficacy. Their taxonomy provides an independent benchmark-level complement to our security-scoped autonomy axis rather than a replacement for record-level decision-authority classification. Halder et al. [30] show that aggregate benchmark scores obscure stakeholder-dependent conclusions: their multi-stakeholder evaluation framework demonstrates that the same model can score 31 points apart depending on whether a CISO, engineering leader, or security researcher lens is applied, confirming that model selection for vulnerability detection is not a single-objective problem.

TABLE 1. OUR COMPARATIVE CODING OF PRIOR SYSTEMATIZATIONS. C=COVERED, P=PARTIAL, A=ABSENT UNDER THE DEFINITIONS USED IN THIS STUDY.

Work	Window-end	Offense	Defense	Loop	Comp./industr.	Contamination
R1 [23]	Aug'24	P	C	A	A	P
R2 [24]	Dec'24	A	C	A	A	P
R3 [25]	Jul'26†	C	P	P (intra-pentest)	A	A
R4 [26]	'25	A	C	A	A	A
R5 [27]	'26	P	P	A	P	A
R6 [28]	Nov'25	A	C	A	P (repo)	P
R7 [4]	Feb'26 preprint‡	C	C	P	C	A
Ours	Jun'26	C	C	C (graded)	C	C (measured+protocol)

†Out-of-window comparator; discussed in related work only.
‡First public preprint is in-window; later publication metadata do not extend the cutoff.

2.3. What Remains Uncovered

No prior systematization joins (i) an autonomy gradient applied to documented behavior across academic, competition, and industrial streams; (ii) loop integration including post-competition redeployment; and (iii) evaluation validity as a measured, protocol-addressable variable. That conjunction is this paper's contribution.

3. Methodology

3.1. Study Design

We conduct a PRISMA-informed systematization with one deliberate structural change: because the subject spans academic papers, competition artifacts, vendor disclosures, and registry records, we maintain **three evidence streams** rather than forcing everything through a single review flow. Stream A (academic literature) follows the classical funnel; Stream B (competition/industry/foundation documents) follows targeted acquisition with quote-level extraction; Stream C (registry records, e.g., NVD entries) follows existence verification. The streams get reported separately and summed only at the top level.

3.2. Search Strategy

Stream A searches used arXiv boolean queries (eight formulations over "large language model" / "language model" / "LLM agent" crossed with vulnerability discovery, detection, penetration testing, CTF, exploit, program repair, cyber reasoning system; total 973 raw hits) and four DBLP queries (214 hits), filtered to January 2024–June 2026 submission dates. Publisher APIs were unavailable; IEEE/ACM/USENIX coverage came from official indexes and citation chaining, a limitation we return to below. Streams B/C used targeted retrieval: official program pages, team/vendor blogs, foundation retrospectives, and NVD. The complete query log with per-query hit counts is Appendix B. Temporal scope follows a record's first public appearance rather than the date of a later revision or venue publication; where later versions fall outside the cutoff, they are used only for bibliographic context unless the specific evidentiary claim is independently supported by an in-window or primary source.

3.3. Selection and Corpus

Screening operated on a deduplicated pool (316 distinct arXiv IDs plus seed/chaining records). Inclusion required peer-reviewed status, influential preprint status, or documented-system evidence relevant to at least one framework cell. The final corpus holds **36 records**: 23 academic works with full texts converted and validated locally, 2 restricted-access surveys positioned via verified metadata, 10 competition/industry primaries with verbatim quote extraction, and 1 registry record. Four candidate records were excluded (wrong identifier, press-only claims, out-of-window scope, unresolvable primary URL). Saturation was tested non-circularly across six novelty axes (new systems, new loop mechanisms, new validity problems, contradictions, new industrial evidence, negative results); two consecutive passes without axis-level novelty stopped inclusion.

Eligibility of general-purpose systems. A system originally designed for general software engineering (e.g., SWE-agent, Agentless, AutoCodeRover) qualifies when the evaluated artifact applies it to a security-specific task: vulnerability discovery, exploit generation, or security patching.

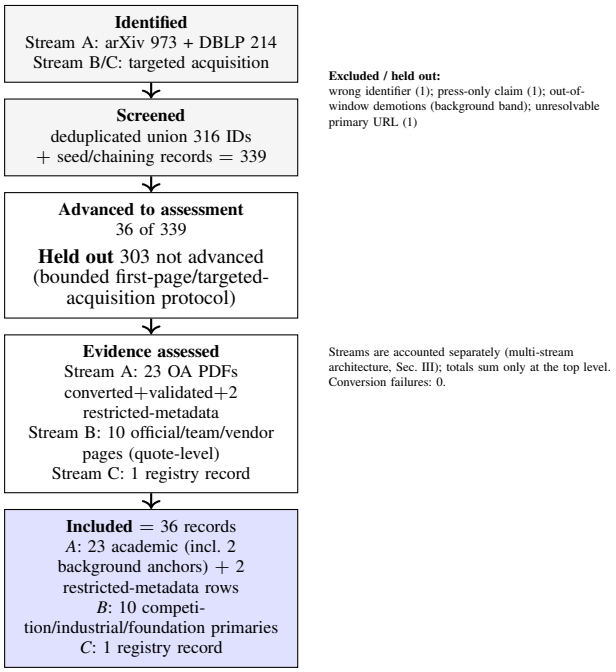

Figure 1. Multi-stream PRISMA flow. Unlike single-corpus reviews, evidence here arrives in three streams with different verification obligations; each stream keeps its own denominators and only the included set is summed (36 records). Full query strings, per-query hit counts, exclusion reasons, and saturation tests appear in Appendices B and the repository artifacts.

The autonomy level reflects the security evaluation, not the system's general capability. AutoCodeRover, for example, is classified A2 because the evaluated artifact applies it to OSS-Fuzz security bugs via CodeRover-S; the general program-improvement system itself does not carry a security autonomy level. This rule ensures the corpus captures the security-relevant frontier without conflating general SWE capability with security-specific deployment.

3.4. Quality Rubric and Evidence Classes

Each record carries reproducibility, baseline-rigor, and artifact-availability scores (0–3), plus evidence class (benchmark, controlled experiment, live enterprise-network experiment, competition, production discovery, incident report, measurement, registry), unit type, access status, and provenance type (PDF+TXT, HTML-extract, metadata-only). Vendor-reported statistics are treated as source-reported unless independently reproduced; third-party reproduction passes are distinguished explicitly where available.

3.5. Classification Protocol

Autonomy (A0–A3) is assigned by the decision-authority test, who chooses the next action?, with A3 also requiring independent planning, execution, validation, and reporting in environment class E2 (production or

production-like external targets; E0=synthetic, E1=realistic-sandboxed). Claims are graded by their weakest missing condition. Loop position uses the set {B1..B4}, recorded as sets for multi-phase systems. The artifact (`evidence/autonomy-loop-assignments.csv`) publishes the record-level assignment fields and classification outputs used for the frozen matrix; benchmarks carry probed phases rather than axis positions, keeping thematic coverage out of the taxonomy.

3.6. Limitations

First-page screening caps per query, publisher-API absence, and English-language sources are documented limitations; the bounded A3 finding remains stable within the documented corpus under the sensitivity scenarios reported in Sec. 4, but cannot exclude undocumented systems outside the observable evidence base. Classification stability: all 36 records were coded for unit type, temporal scope, access, evidence, and loop role; the autonomy×loop taxonomy (Sec. 4) was assigned to the 14 autonomy-assessed records. To check reproducibility, we repeated the classification for all 14 autonomy-assessed records, with six boundary cases (GTG-1002, XBOW, ARTEMIS, Big Sleep, Argusee, AIxCC cluster) getting additional blinded review during initial coding and a blind re-coding pass. The two passes yielded consistent classifications, though intermediate uncertainty concentrated on environment-class boundaries (E1 vs. E2); GTG-1002 remains unresolved because the available incident documentation cannot support a definitive autonomy level. The weakest-missing-condition rule preserved that uncertainty and did not change any assigned record's autonomy level. We report this as evidence that the taxonomy produces stable classifications when applied by the same coder (a within-coder stability check); a formal inter-rater study with independent domain-expert coders remains future work. Additional methodological constraints include: (i) our autonomy classification requires judgment at boundary cases, which we bound with the weakest-missing-condition rule but do not eliminate; (ii) vendor-reported statistics travel without independent verification in most industrial records; (iii) the corpus skews toward organizations that publish, leaving commercial pentesting vendors, state programs, and internal red teams structurally invisible; and (iv) rapid model churn means any conclusion phrased about "LLMs" generally is hostage to a moving baseline. We address (i) by publishing every record's assignment fields in the artifact CSV, (ii) by explicitly distinguishing source-reported from independently reproduced evidence where available, and (iii)–(iv) by phrasing findings about *records and their conditions*, not about model classes.

4. The Autonomy × Loop Framework

4.1. Unit of Analysis

Records are typed before classification: only *systems*, *clearly specified experiments*, and *incident documentations* receive autonomy levels; benchmarks carry *probed loop phases*; datasets carry task domains; surveys and systematizations carry the *documented systems* they describe. This prevents a paper about autonomous systems from becoming an autonomous-system datapoint. Our corpus contains one such systematization of AIxCC [4], and it is classified *n/a* on both axes.

4.2. Axis A: Decision Authority

The autonomy gradient is operationalized by one question: **who chooses the next action?** *A0 assistive*: no acting system; a prompted model responds. *A1 pipeline-orchestrated*: a fixed controller determines the workflow; the model fills stages. Tool use does not imply A1→A2, a system invoking ten tools under fixed control remains A1. *A2 task agent*: the model dynamically selects actions and tools within a bounded task environment. *A3 autonomous production hunter*: independent planning, execution, validation, and reporting (*all four verbs*) in environment class E2, i.e., production or production-like external targets (E0=synthetic/benchmark; E1=realistic but sandboxed or competition). Environment class is determined by the target environment, not the experimental design: AIxCC is E1 (forked repositories, inserted bugs), while ARTEMIS is E2 (corporate-scale network, production-like infrastructure). Two corollaries do heavy lifting: *no human interaction* ≠ A3, since AIxCC CRSs satisfied execution autonomy inside forked challenge repositories; and the *weakest-missing-condition rule*: every autonomy claim is graded by its weakest unmet condition, never by headline percentages, which is how "fully automated" XBOW classifies A2 (policy-mandated pre-submission review) and how the GTG-1002 incident remains *unresolved* rather than A3, where evidence is insufficient to assign a definitive autonomy level.

4.3. Axis B: Loop Position

B1 discovery; B2 triage/validation/exploitation; B3 patching/repair; B4 redeployment/regression/monitoring. Axis B is a set: multi-phase systems are recorded as sets (e.g., AIxCC CRSs $\{B1, B2, B3\}$) and drawn with neutral connectors denoting documented coverage, not inferred sequencing.

4.4. The Map

Figure 2 places the in-window corpus; Table 2 lists all 36 records individually. The A2 row is populated across all three loop phases; the A3 row is empty under the operational definition; B4 exists only as loop-transition documentation tied to documented systems. Counts are reconciled against the assignment CSV and the record inventory; the table is a frozen audit view rather than a generated data product.

4.5. Migration, 2024 → June 2026

Early in-window records are dominated by benchmarked/prompted capability evaluation, while the system-

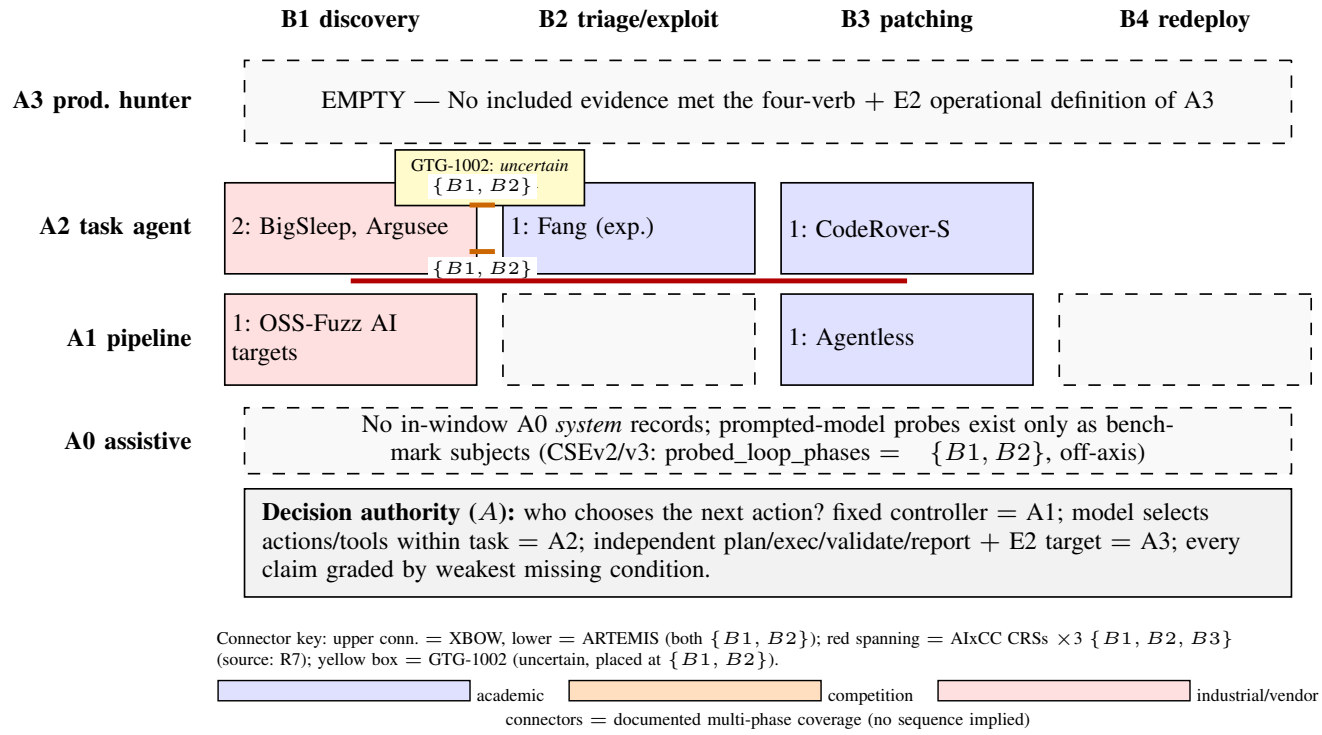

Figure 2. Autonomy × loop map over autonomy-bearing records (in-window corpus). Only records whose unit is a system, a clearly specified experiment, or an incident documentation receive A-levels (12 records, plus 2 general-SE comparators); benchmarks report *probed loop phases*, datasets report *task domains*, and systematization papers report *documented systems*. Axis B is the set $\{B1..B4\}$; multi-phase operation uses neutral connectors denoting documented coverage. The A3 row is empty under the four-verb + E2-environment definition; the nearest candidate (GTG-1002) remains uncertain under the weakest-missing-condition rule.

bearing evidence shifts toward A1 pipeline automation (OSS-Fuzz's LLM stages; Agentless) and A2 task agents, arriving first as seeded production research (Big Sleep, November 2024), then proliferating across independent industrial (Argusee, May 2025; XBOW, June 2025), competition (AIxCC finals, August 2025), and live enterprise-network experimental settings (ARTEMIS). We emphasize what this phrasing avoids: our corpus contains no in-window A0 systems, so claims of a field-wide "A0→A1→A2 migration" would outrun the evidence; what migrated is where humans sit (boundaries: seeds, entry points, review, reporting) and what gets scored. The 2026 record adds consolidation: four systematizations, an infrastructure-risk cluster around agent tool protocols, and the first institutionalized B4 documentation.

A3 robustness. No included evidence met our operational definition of A3. The sensitivity analysis makes the counterfactuals explicit: with independent planning/target selection (P), execution (X), validation (V), reporting (R), and an E2 target (E), the baseline predicate is $P \wedge X \wedge V \wedge R \wedge E$. Relaxing R alone admits XBOW, while relaxing P or E alone admits none; relaxing P, R, and E together admits XBOW and the AIxCC CRS cluster under that altered rule. These are counterfactual qualifiers, not A3 findings, and the core conclusion remains bounded to the strict definition. The weakest-missing-condition rule prevents a system from qualifying under the strict rule merely because of a headline autonomy claim; the remaining blockers are documented per candidate in Appendix D.

5. Discovery: From Capability Probes to Production-Oriented Agents

Reading the in-window corpus through the decision-authority test yields a discovery landscape that separates cleanly into pipeline automation (A1) and task-level agents (A2), with no A0 *system* records: prompted-model capability probes persist only as benchmark subjects (Sec. 4), not as operational discoverers.

5.1. A1: Pipeline-Orchestrated Discovery at Ecosystem Scale

The most extensive automated discovery effort in our corpus is Google's AI-augmented OSS-Fuzz program. Its LLM executes the first four steps of the developer fuzzing workflow, namely drafting fuzz targets, repairing compilation and runtime faults, running campaigns, and triaging crashes into driver-vs-project attributions, while step five

TABLE 2. COMPLETE RECORD-LEVEL CLASSIFICATION (36 RECORDS). A = AUTONOMY LEVEL; B = LOOP-POSITION SET; T = TEMPORAL SCOPE. ONLY SYSTEMS, CLEARLY SPECIFIED EXPERIMENTS, AND INCIDENT DOCUMENTATION BEAR A-LEVELS; BENCHMARKS REPORT PROBED PHASES (^{p}); DATASETS REPORT TASK DOMAINS; SYSTEMATIZATIONS ARE N/A WITH DOCUMENTED SYSTEMS. PER-RECORD BEHAVIORAL EVIDENCE: EVIDENCE/AUTONOMY-LOOP-ASSIGNMENTS.CSV.

Record	Year	T	A	B-set / role
(i) In-window autonomy-bearing records (14)				
bigsleep2024	2024	in	A2	$\{B1\}$
argusee2025	2025	in	A2	$\{B1\}$
fang2024one-day	2024	in	A2 (exp.)	$\{B2\}$
sweagent2024	2024	in	A2	$\{B3\}$
autocoderoever-2024	2024	in	A2	$\{B3\}$
ossfuzz-aifixing-2024	2024	in	A2	$\{B3\}$
xbow2025	2025	in	A2	$\{B1{+}B2\}$
artemis2025	2025	in	A2 (exp.)	$\{B1{+}B2\}$
r7-CRS-cluster recs:				
tob-buttercup2025	2025	in	A2	$\{B1{+}B2{+}B3\}$
teamatlanta-sqlite2024	2024	in	A2	$\{B1{+}B2{+}B3\}$
teamatlanta-afc2025	2025	in	A2	$\{B1{+}B2{+}B3\}$
gtg1002-2025	2025	in	uncertain	$\{B1{+}B2\}$
ossfuzz-levelingup-2024	2024	in	A1	$\{B1\}$
agentless2024	2024	in	A1	$\{B3\}$
(ii) Background anchors, excluded from primary counts (2)				
intercode2023bg	2023	bg	A1	$\{B2\}$
csev12023bg	2023	bg	A0	$\{B1\}$
(iii) Loop-transition documentation, B4 tied to systems (2)				
openssfretro2026	2026	in	(A2 CRS cluster)	documents $B3{\rightarrow}B4$
bloggoogle-summer2025	2025	in	(A2 BigSleep)	documents $B1{\rightarrow}B4$ redeploy
(iv) Off-axis / context records (18)				
cybench2024	2024	in	n/a	benchmark; probed $\{B2\}^p$; embedded A2-class scaffold
nyuctfbench2024	2024	in	n/a	benchmark; probed $\{B2\}^p$; embedded A2-class scaffold
csev22024	2024	in	n/a	benchmark; probed $\{B1, B2\}^p$
csev32024	2024	in	n/a	benchmark; probed $\{B1, B2\}^p$
primevul2024	2024	in	n/a	dataset; task domain $B1$ detection
rissebohme-wrongexam2024	2024	in	n/a	methodological critique
mcpeco2025	2025	in	n/a	MCP ecosystem measurement
mcpgov2025	2025	in	n/a	MCP governance
mcpspec2026	2026	in	n/a	MCP spec analysis
mcppoison2026	2026	in	n/a	MCP tool poisoning
nvdcve20256965	2025	in	n/a	registry record (CVE-2025-6965)
r1slr2024 (R1)	2024	in	n/a	survey (SLR)
r2slr2024 (R2)	2024	in	n/a	survey (SLR)
r4csur2025 (R4)	2025	in	n/a	survey (paywalled; metadata)
r5usenix26agentic (R5)	2026	in	n/a	survey (agentic-AI security)
r6tosem2026 (R6)	2026	in	n/a	survey (263 studies; repo open)
r7sokaixcc2026 (R7)	2026	in	n/a	survey(systematization); documents Atlantis, Buttercup, Theori-SRS, RoboDuck, FuzzingBrain, Lacrosse, Shellphish
r3survey2026 (R3)	2026	OUT	n/a	survey comparator (out-of-window)

Totals: 14 autonomy-assessed ($A1{\times}2$, $A2{\times}11$, uncertain$\times 1$) + 2 background + 2 loop-transition + 18 context = 36 records.

(fixing) was explicitly future work at announcement time and an agent-based architecture was listed as a roadmap item, anchoring its A1 classification [31]. The reported yield is material: 26 new vulnerabilities across OSS-Fuzz projects, including CVE-2024-9143 in OpenSSL, assessed by Google as likely present for two decades and "wouldn't have been discoverable with existing fuzz targets written by humans" [31]. Two features matter for our framework. First, the workflow controller, not the model, fixes the stage sequence, the defining A1 property. Second, triage automation remains pre-human-review: the authors state the goal of reaching confidence "not requiring human review," implying review persisted at announcement time [31].

5.2. A2: Seeded Variant Analysis in Production Code

Project Zero's Big Sleep provides one of the earliest documented production-oriented A2 discoveries in our corpus: an exploitable stack buffer underflow in

SQLite's `seriesBestIndex`, triggered through the `generate_series` virtual table under a ROWID constraint, found by variant analysis over a human-curated pool of recent commits using Gemini 1.5 Pro [1]. Three qualifiers discipline any capability claim. The seed pool was human-selected; the run's target was inspired by an AIxCC finding (Sec. 8); and the authors themselves note the result is experimental and that "a target-specific fuzzer would be at least as effective" in general [1]. The built-in baseline is instructive: AFL failed to rediscover the bug after 150 CPU-hours even with a corpus pre-seeded with the required keywords, and the OSS-Fuzz harness lacked the `generate_series` extension entirely , the comparison points to harness and environment coverage as an important limiting factor; the 150 CPU-hour result does not isolate compute from harness design. No CVE was assigned: the bug was fixed pre-release [1].

5.3. A2: Entry-Point-Assisted Auditing

DARKNAVY's Argusee distributes auditing across Manager, Auditor, and Checker agents backed by an LSP-based code reader, and DARKNAVY reports that Argusee discovered and analyzed CVE-2025-37891, a vendor-characterized arbitrary kernel heap overflow in the Linux USB MIDI2-to-UMP conversion path, alongside fifteen further vendor-reported flaws in projects such as GPAC and GIFLIB [32]. Its own text, however, fixes the autonomy boundary: Argusee "is not intended to completely replace manual auditing or to discover vulnerabilities from scratch," operating from *human-supplied entry points*; exploitation of CVE-2025-37891 to root on Arch Linux was performed by the human team, and Reproducer/Exploit agents are listed as future work [32]. Argusee also reports perfect scores on CyberSecEval 2 buffer-overflow cases, a saturation datum we return to in Sec. 9.

5.4. A2: Black-Box Platform Operation

XBOW operated as a commercial submission engine across public and private HackerOne programs: an LLM-assisted targeting layer parsed program policies, scored assets, and deduplicated infrastructure, while internal "validators" re-verified findings before submission [2]. The vendor reports $\sim$1,060 submissions of which 130 were resolved and 303 triaged, with 208 duplicates and 209 informative outcomes, roughly 39 percent of submissions falling into duplicate/informative categories rather than new/resolved findings, a metric-definition problem for leaderboard-style counting rather than simple noise . Including a previously unknown Palo Alto GlobalProtect VPN issue affecting 2,000+ hosts, for which no CVE identifier is given [2]. The decisive classifier for our taxonomy is the platform rule: "our security team reviewed them pre-submission to comply with HackerOne's policy on automated tools" [2]. Independent reporting fails, so XBOW is A2 regardless of marketing language.

TABLE 3. DISCOVERY-SIDE COMPARISON ACROSS AUTONOMY LEVELS. E-CLASS PER PROTOCOL V4; VALIDITY FLAGS USE THE V1–V5 TAXONOMY OF SEC. 9. OPERATIONAL STATISTICS ARE SOURCE-REPORTED; THEIR PROVENANCE VARIES ACROSS VENDOR DISCLOSURES, RESEARCH REPORTS, AND CONTROLLED EXPERIMENTS.

Record	*A*/E	Human locus	Metric	Baseline	Validity risks
OSS-Fuzz AI [31]	A1/E2	report review	26 vulns; CVE-2024-9143	prior harnesses	V5 triage counts
Big Sleep [1]	A2/E2	curated seeds	n=1 find + repro	AFL 150 CPU-h fail	V1 model preknowledge
Argusee [32]	A2/E2	entry points	CVE-2025-37891; 15 claims	CSEv2 100% (sat.)	self-adjudication
XBOW [2]	A2/E2	mandated review	funnel stats; rank	leaderboard	unaudited dashboard
ARTEMIS [33]	A2/E2	comparison arm	first-10h score	professionals	single environment

5.5. A2 Under Experiment: Production Networks

The ARTEMIS study was first posted as an arXiv preprint in December 2025; the local evidence copy is v2 dated March 2026. It places seven agent instances (six existing scaffolds plus ARTEMIS) against ten professionals on the same large research-university enterprise network ($\sim$8,000 hosts, 12 subnets), scoring the first ten hours of sixteen-hour runs [33]. We use it as environment-class E2 evidence that agents can operate on realistic infrastructure; comparative performance figures are treated separately because the study's evaluation design does not isolate general agent capability from scaffold and task conditions. Liu et al. [34] extend the A2 paradigm to consensus-protocol auditing, where LLM agents autonomously detect bugs in production-level blockchain implementations through code decomposition and adversarial verification, a domain-specific demonstration that A2 task-agent patterns generalize beyond traditional software stacks.

5.6. Cross-Cutting Comparison

Table 3 consolidates the discovery records. Reading down the autonomy column, the pattern consistent with I1 is cross-sectional rather than longitudinal: every record retains an identified human locus (seed curation, entry points, mandatory review, target selection), and what we observe is that human involvement increasingly appears at the boundaries of the autonomous workflow, including seed selection, target selection, entry-point specification, and final reporting, rather than on each individual tool action, while evaluation, funding, and discourse increasingly score systems by their degree of autonomous operation.

6. Exploitation and Validation: The AIxCC Case Study

The Defense Advanced Research Projects Agency's Artificial Intelligence Cyber Challenge (AIxCC) is the only included setting in our corpus where discovery, proof-of-vulnerability generation, and patching were integrated, scored, and operated without human intervention, and it

is best analyzed not as a benchmark but as a deliberately constructed offense↔defense instrument whose rules, incentives, and aftermath document how the field's unit of progress was administratively redefined (I1). Beyond competition settings, recent benchmarks such as CVE-Bench [35] evaluate LLM agents on real-world web application vulnerabilities, reporting that agent frameworks exploit up to 13% of critical-severity CVEs in sandboxed environments under the benchmark's one-day setting, where a high-level vulnerability description is supplied to the agent [35]. Information conditions can change exploitation outcomes sharply in separate controlled experiments [6]. Multi-agent red-teaming frameworks like Co-RedTeam [36] demonstrate that orchestrated discovery and exploitation stages, grounded in execution feedback and long-term memory, achieve over 60% success rates on exploitation tasks across multiple security benchmarks; those exploit tasks can provide the target codebase together with a description of the specific vulnerability to be reproduced. CyberGym [37] further scales evaluation to real-world vulnerabilities at industrial scale, providing evidence on the computational costs and failure modes of agentic exploitation.

6.1. Design and Incentives

Announced in 2023 with ARPA-H participation and support from Anthropic, Google, Microsoft, and OpenAI, the challenge funneled 42 semifinal entrants down to seven finalists, each receiving $2M to harden a Cyber Reasoning System (CRS) for a final round "set up to mimic a real world CI/CD pipeline" [38]. Scoring rewarded finding vulnerabilities, proving them via PoVs, and applying correct patches, with speed and accuracy modifiers; "human interaction was strictly prohibited" during scored operation [39]. Two incentive details matter for interpretation. Patching was worth three times discovery (6 vs. 2 points), making B3 capability the rational investment [40]; and accuracy modifiers penalized incorrect patches nonlinearly; Team Atlanta reports its PoV-validated 91.27% patch accuracy yielding a modifier of $1-(1-0.9127)^4 = 0.9999$ and reports Theori's PoV-free 44.4% as 0.9044 [41]. Applying the displayed formula independently to 0.4444 gives 0.9047, so we preserve 0.9044 only as a source-reported value. The scoring mathematics thus priced patch-quality adjudication directly into strategy.

6.2. What the Final Round Measured

Trail of Bits describes the scored structure as 48 challenges across 23 open-source repositories in its participant account [39]. The later R7 systematization counts the same 48 challenge projects across 24 repositories; we retain each figure within its respective source and accounting scope [4]. The CRSs also surfaced real, non-synthetic vulnerabilities; Team Atlanta reports six C/C++ and twelve Java bugs found collectively during the final, including a SQLite zero-day first seen at semifinals [41]; and Trail of Bits demonstrated a PoV for a vulnerability that had *not* been inserted into any challenge [39]. Real-bug discovery did not score; Team Atlanta footnotes that their autonomous find-and-patch of a real SQLite bug during semifinals earned nothing competitively [40]; yet it is precisely this unscored spillover that connects the competition to the production ecosystem. DARPA's official closeout reports the remaining aggregates: competitors' systems discovered 54 unique synthetic vulnerabilities across 63 final challenges (86%) and patched 43 (68%), alongside 11 patches for the real vulnerabilities [3]; DARPA's scoring guide separately confirms both the three-to-one patching incentive and the $8.5M final prize pool [42]. The closeout also carries an editor's note correcting its own earlier count of 70 synthetic vulnerabilities to 63, an instance of administrative record instability we separate from measurement validity throughout (Sec. 9).

6.3. CRS Design Space

The finalist architectures span the design axes our framework distinguishes. *Atlantis* (Team Atlanta) ran N-version programming across orthogonal sub-CRSs (C, Java, multilanguage, patching, SARIF), ensemble fuzzing and concolic execution alongside LLM agents, and an explicit integration taxonomy (LLM-augmented, LLM-opinionated, LLM-driven) with smaller models often outperforming larger or reasoning models for patch tasks; its validation oracle reruns PoVs against patched builds while acknowledging that MTE/PAC recompilation or broad `catch(Exception)` wrappers can suppress PoVs without semantic repair [41]. *Buttercup* (Trail of Bits) took the opposite bet: deterministic, expertise-decomposed workflows with LLMs only where classical tools fail, using exclusively non-reasoning models, achieving lower reported cost per competition point at $181 per point (versus Atlanta's $263) on 28 vulnerabilities and 19 patches across 20 CWE classes with $>90\%$ accuracy [4], [39]. *Theori* demonstrated purely static, PoV-free patching, trading accuracy for robustness to absent proofs [41]. Team-level compute costs are independently consistent across our two sources: totals of $103.3K (Atlanta), $39.6K (Trail of Bits), and $31.8K (Theori) appear in both the systematization analysis and the participant account [4], [39].

6.4. Brittleness as Environment Evidence

The most cited lesson inside the winning team is a string-matching heuristic: Atlantis skipped patch generation for paths containing "fuzz," and organizers' "ossfuzz" directory prefixing nearly disabled the system hours before the deadline: "here we were, building an autonomous CRS powered by state-of-the-art AI, nearly defeated by a string matching bug" [41]. For our validity taxonomy this is textbook V4: benchmark-specific heuristics functioned as load-bearing components, invisible until the environment shifted.

6.5. Architecture Lessons and the Epistemics of A3

The finalist designs instantiate three distinct philosophies about where intelligence belongs in a CRS. Atlantis distributes it across N-version programming: independent subsystems for C, Java, multilanguage discovery, patching, and

SARIF bundling, deliberately orthogonal so a failure in one does not cascade; ensembling was explicitly framed as the engineering answer to LLM non-determinism ("what the ML community calls hallucination, systems engineers call non-determinism") [41]. Buttercup concentrates expertise in deterministic decomposition, integrating LLMs "only where traditional tools fall short," augmenting libFuzzer and Jazzer with LLM-generated test cases behind a multi-agent patcher with separation of concerns, and attributed its efficiency to this architecture rather than model scale: $181 per competition point using exclusively non-reasoning models, versus $263 for the winner [39]. Theori bet on removing the runtime oracle entirely, producing correct patches by pure static analysis at 44.4% accuracy, a strategy the accuracy-modifier formula priced heavily but did not disqualify [41]. Two quantitative facts refine any capability narrative. First, smaller models "often outperformed larger foundation models and even reasoning models" for patch generation [41], echoing scaffold-over-scale findings elsewhere in our corpus. Second, validation infrastructure capped agent counts: re-verifying one nginx patch takes ten-plus minutes, so Atlantis ran six patching agents rather than sixty, illustrating that, in this architecture, build-and-test throughput became a practical bottleneck on B3 scaling. The competition also yields its cleanest end-to-end A2 datapoint. During semi-finals, Atlantis autonomously identified and patched a real SQLite FTS5 vulnerability (an off-by-one read in the trigram tokenizer reaching a NULL dereference) in roughly fifteen minutes spanning build, patch generation, iteration, and validation; the maintainer's subsequent fix (commit e9b919d5) extended the pattern to sibling tokenizer functions and was itself revised in a follow-up commit [40]. Discovery and repair here were performed by the system; the exploit-to-root-privilege demonstration afterward was not. What AIxCC can and cannot establish follows directly. It demonstrates integrated $\{B1, B2, B3\}$ operation with execution autonomy under active prohibition of human input, the strongest documented autonomy evidence in our corpus. It does not satisfy the paper's A3 predicate: challenges are forked repositories carrying inserted bugs (environment class E1, not E2), real-bug discovery went unscored, and DARPA has not released the telemetry that would let outsiders audit either claim [39]. Post-competition triage sharpened the point rather than resolving it: Ada Logics reproduced twenty-seven candidate real issues, while other findings dissolved into already-fixed code, project threat-model exclusions, or known OSS-Fuzz reports [38]. Outcome adjudication, not discovery, was the binding difficulty.

6.6. Interpretive Discipline

Three boundaries govern what AIxCC evidence can support. First, E-class: competition forks with inserted bugs are environment E1; substantial autonomy under prohibition of human interaction still fails A3's production-environment condition. Second, outcome definitions: "discovered" means a scored synthetic-vulnerability PoV, "patched" means oracle-passing; the oracle-gaming taxonomy above bounds what "correctly patched" certifies. Third, record reliability: figures here cite corrected primaries (DARPA close-out, participant blogs cross-checked against the systematization), never press intermediaries. With those bounds, AIxCC demonstrates that integrated $\{B1{+}B2{+}B3\}$ operation at A2 is achievable within one engineering generation, and its aftermath, examined in Sec. 8, shows the loop continuing through funded redeployment rather than ending at a scoreboard.

7. Patching: The B3 Stream and Its Blind Spots

For a title promising discovery, exploitation, *and* patching, B3 is where the in-window corpus is thinnest, and where its structure is therefore most informative. Autonomous repair evidence arrives through two independent channels: a general software-engineering agent lineage repurposed for security, and the competition crucible of Sec. 6.

7.1. The Agent Lineage

Three 2024 systems establish the design contrast that our decision-authority test formalizes. SWE-agent demonstrates that interface design, the agent–computer interface through which the model issues file, search, and edit actions, governs autonomous issue-resolution performance [43]. AutoCodeRover operationalizes iterative, program-structure-aware localization and repair, letting the model decide which repository context to request next [44]. Agentless deliberately inverts the design point: a fixed localize→repair→validate pipeline with no dynamic action selection matches or beats agentic baselines at lower cost [7]. The security specialization of this lineage, CodeRover-S, repurposes AutoCodeRover against OSS-Fuzz vulnerabilities and states the distinction explicitly: "having autonomy in the LLM agent is useful for successful security patching, as opposed to approaches like Agentless where the control flow is fixed" [45]. Its mechanism satisfies our test at the action level; the model invokes AST search, file navigation, compilation, and test-execution tools, gathering failure information before committing patches. Reported efficacy on real OSS-Fuzz vulnerabilities remains far from saturation (52.4% on a curated set under the paper's conditions) [45], and validation is sanitizer/test-based rather than exploit-aware, precisely the gap AIxCC's oracle-gaming taxonomy names. CVE-Genie reproduces approximately 51% of CVEs published in 2024–2025 with verifiable exploits at an average cost of $2.77 per CVE [46], demonstrating the feasibility of automated vulnerability reproduction at scale. Recent surveys map the broader scope of LLM-based automated program repair [47], identifying four paradigms: fixed-workflow, single-agent, multi-agent, and general-purpose code agents. On the security-specific repair side, tree-of-thought prompting with execution-log feedback achieves substantially higher patch correctness than direct

generation on real-world vulnerability datasets [48], suggesting that structured reasoning over repair candidates rather than single-shot generation is an important design choice for B3 systems. LLM4CVE [49] extends this line with an iterative pipeline that combines traditional bug repair methods with LLM techniques, demonstrating automated vulnerability repair in critical system software with minimal human input. Beyond competition settings, Nong et al. [50] demonstrate that vulnerability semantics reasoning combined with adaptive prompting enables automated patching of zero-day vulnerabilities without test inputs or exploit evidence, achieving up to 28.33% F1 improvement over non-LLM baselines on 97 zero-day vulnerabilities. These systems establish that B3 capability now spans from pipeline-orchestrated repair (Agentless) through iterative agentic workflows (AutoCodeRover, CodeRover-S), yet validation of semantic security correctness beyond replaying known exploit inputs remains unevenly evaluated across the literature. Two further findings qualify this picture. Wu et al. [51] demonstrate how a known patched instance can be abstracted into a recurring error pattern and searched at repository scale: across the Linux kernel their system surfaced more than 22,000 candidate issues, with 246 of 400 manually inspected samples confirmed as valid issues; separately, their recurring-pattern identification evaluation reports 92.2% precision under its own evaluation protocol, showing that variant analysis scales further than the n=1 result our A2 discussion initially suggests. Meng et al. [52] provide a systematic comparison of LLM-based bug-fixing systems, showing that fault localization and the interaction between issue reproduction and repair as major determinants of repair success, indicating that additional agentic interaction does not uniformly improve outcomes, a finding with direct implications for the B3 autonomy gradient. Adjacent discovery-audit systems such as RepoAudit [53] demonstrate that comparable agentic control patterns also extend upstream into repository-level vulnerability analysis. OpenAnt [54] combines repository-scale decomposition with adversarial verification and dynamic testing, providing another example of a verification loop applied to repository-scale vulnerability analysis. Li et al. [55] provide the first comprehensive empirical comparison of five LLM-based detectors against traditional static analyzers at project scale, finding that while LLM-based tools uncover unique vulnerabilities missed by traditional tools, both approaches suffer from very high false discovery rates, with shallow interprocedural reasoning and misidentified source/sink pairs as primary failure modes, a result that directly qualifies the A2 systems in our corpus, which operate at the same project-scale detection level but have not been independently evaluated against this failure profile. Charoenwet et al. [56] extend this line with AgenticSCR, an autonomous agentic secure code review system for immature vulnerabilities that combines LLM reasoning with static analysis tools in a pre-integration setting, demonstrating that agentic code review can provide earlier security feedback than traditional deferred assessment.

7.2. What the Competition Taught About Repair Quality

AIxCC provides the corpus's largest competition-based stress test of automated patching. Its lessons map directly onto validity axis V5: PoV re-execution establishes behavior only for the tested proof; it does not certify semantic correctness; recompilation with MTE/PAC or broad exception wrapping can pass without repairing root causes, so teams built additional oracles (optional `test.sh`) and accuracy modifiers became the binding score component [41]. Patch correctness is itself iterative and review-dependent: the SQLite maintainer's own fix for the FTS5 bug initially added redundant checks to sibling tokenizer functions and required revision in a follow-up commit [40]. Ensemble patching emerged as a prominent mitigation strategy among the AIxCC architectures for LLM non-determinism; Atlantis capped its patching agents at six not by principle but because validating one nginx patch takes ten-plus minutes [41], an infrastructure constraint on B3 scaling that benchmark-level success rates do not capture.

7.3. Blind Spots

Five gaps bound current evidence. (i) *Semantic exploit-aware validation*: several systems, including CodeRover-S and PatchEval-style evaluations, replay known exploit inputs or PoCs against candidate patches. What remains sparsely documented outside the AIxCC ecosystem is broader validation that tests whether exploitability has been eliminated beyond the originally observed proof, for example through alternate exploit inputs, semantic analysis, or adversarial mutation [45], [57]. (ii) *Adversarial inputs to repair*: Przymus et al. [58] demonstrate that adversaries can craft bug reports causing LLM-based repair systems to introduce new vulnerabilities while ostensibly fixing the reported bug: in their evaluation, 90% of crafted reports triggered attacker-aligned patches, while existing input and output defenses remained substantially incomplete, a trust-boundary violation the manuscript's B3 discussion does not address. (iii) *Regression and redeployment*: B4 exists in our corpus solely as foundation-documented ecosystem transition (OpenSSF SIG hosting, $200K-per-team integration funding, Team Atlanta's $2M prize recycling into continuous hunting [38], [39], [41]), never as an academic evaluation target with regression metrics. (iv) *Attacker adaptation*: no included work studies adversaries reacting to deployed auto-patchers; the offensive side of B3 is absent. (v) *Interaction with discovery*: patching and discovery are evaluated as separate competencies everywhere except inside CRS pipelines; whether autonomous discovery changes patch demand or maintainer workload at ecosystem scale is unmeasured (DARPA has not yet released the telemetry that would permit it [39]). These gaps seed the roadmap experiments of Sec. 11.

8. The Co-Evolution Loop: Graded Evidence

Our second insight concerns coupling between offense and defense. We grade every candidate edge on a three-level scale: *documented response* (an explicit, sourced statement or traceable institutional flow), *plausible coupling* (mechanism-reasonable temporal adjacency), and *temporal association* (chronology alone), and claim only what the grades support: multiple documented edges exist; field-wide causal co-evolution does not follow from them.

8.1. Documented Response Edges

(D1) Competition → industrial research program. Project Zero's Big Sleep post opens its target rationale by citing the challenge directly: "Earlier this year at the DARPA AIxCC event, Team Atlanta discovered a null-pointer dereference in SQLite, which inspired us to use it for our testing" [1]. This is the corpus's cleanest causal edge: named source, stated mechanism, dated. **(D2) Maintainer-side variant response.** After Atlantis reported the FTS5 trigram off-by-one, the SQLite maintainer patched not only the reported function but sibling tokenizer code paths sharing the pattern, then revised his own patch in a follow-up commit after noting redundancy [40]. Human variant analysis triggered by an AI finding. **(D3) Institutional/funding flows.** The loop acquired governance: DARPA and ARPA-H created transition incentives of up to $200K per team to integrate CRSs into critical software [3]; Team Atlanta recycled half its $4M prize ($2M) into continuous autonomous hunting at Georgia Tech's SSLab with OpenAI contributing API credits [41]; OpenSSF chartered a Cyber Reasoning Systems SIG and hosts OSS-CRS and FuzzingBrain, whose post-competition operation is reported at 62 vulnerabilities across 26 projects (43 maintainer-confirmed, 36 patched upstream) for FuzzingBrain and 25 vulnerabilities across 16 projects for OSS-CRS [38]. These are foundation-relayed, team-reported figures; the funding and governance mechanisms provide evidence that competition-derived capabilities persisted beyond the scored event. **(D4) Threat-seeded defensive disclosure.** Google's CVE-2025-6965 disclosure couples its Threat Intelligence unit to Big Sleep: the flaw was "known only to threat actors," and the company asserts an imminent exploit was cut off: "we believe this is the first time an AI agent has been used to directly foil efforts to exploit a vulnerability in the wild" [59], [60]. The threat-intelligence premise is unverifiable and the foiling counterfactual; we cite it as documented defensive process, not verified attack prevention.

8.2. Plausible Coupling

(P1) Attacker-side adoption of agentic operation within roughly twelve months of defender agentic publicity: Anthropic reports (as a single-source threat-intelligence assessment) that GTG-1002 used Claude Code for 80–90% of an espionage campaign's execution: "the AI autonomously discovered vulnerabilities in targets selected by human operators" [5], [61]. Directionality is unverifiable without adversary telemetry; shared capability tide remains a live alternative.

8.3. Temporal Association Only

Benchmark releases trailing industrial demos, press arms-race narratives, and leaderboard movements populate this tier; per our claim ledger they carry no inferential weight here.

8.4. Falsification Test and Record Reliability

The shared-cause objection (frontier-model releases independently advancing both sides) survives globally: it is defeated only for edges D1–D2, where stated mechanisms precede any model-release timing argument. We therefore do not generalize from the AIxCC–Google hub to the field. Record reliability itself becomes evidence, once accounting scopes are labeled precisely: $29.5M denotes the cumulative program prize commitment announced in 2023 [62], $8.5M denotes the final competition pool per DARPA's scoring guide [42], and OpenSSF reports $30.5M ultimately distributed across both rounds [38], distinct boundaries, not competing totals for one quantity. Separately, DARPA's own closeout corrects its earlier synthetic-vulnerability count from 70 to 63 [3]: an administrative record correction showing that even authoritative primary sources require version- and date-aware verification.

8.5. Institutional Response

Between one-off response and field-level causation sits a grade our corpus documents richly: institutionalization. Three funding and governance flows carry competition-derived capability past the scoreboard. DARPA and ARPA-H created transition incentives of up to $200K per finalist team to integrate CRSs into critical software [3]. Team Atlanta recycled half of its $4M prize into continuous autonomous hunting of OSS-Fuzz projects at Georgia Tech's SSLab, with OpenAI contributing API credits [41]. And OpenSSF chartered a Cyber Reasoning Systems Special Interest Group that now hosts OSS-CRS and FuzzingBrain, relaying post-competition yields: sixty-two vulnerabilities across twenty-six projects for FuzzingBrain (forty-three maintainer-confirmed, thirty-six patched upstream), twenty-five vulnerabilities across sixteen projects for OSS-CRS, and twelve kernel-related plus ten user-space zero-day findings for 42-b3yond-6ug [38]. These counts remain team-relayed; the institutions, budgets, and standing governance are independently observable. Google's parallel commitment (deploying Big Sleep against widely used open-source projects [59]), extends the same pattern from competition lineage to industrial program. On the speculative side of the ledger sits attacker-side adoption. The strongest candidate is GTG-1002: a framework wrapping Claude Code with MCP

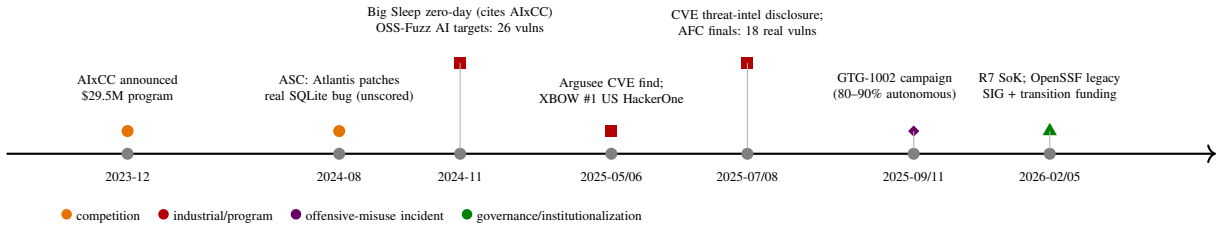

Figure 3. Capability-and-coupling timeline for the graded edges of Sec. VIII. Colors separate evidence classes; every event carries a primary source in Appendix C. The sequence visualizes I2's documented edges — most notably the AIxCC→Big Sleep inspiration link — without implying field-wide causal closure.

tool access, jailbroken through task decomposition and fabricated defensive-employment framing, executing reconnaissance, exploit drafting, credential harvesting, and exfiltration with humans selecting targets and retaining strategic authorization at critical escalation points [5], [61]. The incident remains an unresolved, single-source autonomy assessment. Its grade stays *plausible coupling*: no adversary-side telemetry exists to date whether defender publications caused, accelerated, or merely coincided with the technique transfer. A concrete falsifier structures future work: if adoption-lag measurement finds agentic offensive techniques appearing no earlier after major defender disclosures than baseline diffusion rates would predict, response-coupling loses its offensive half and I2 narrows to defender-internal loops.

8.6. Summary

The loop is real where it is documented: a competition generated tooling whose inspiration seeded an industrial program (D1), whose findings triggered maintainer responses (D2), whose participants now operate continuously under foundation governance (D3), and whose defender capability claims now interleave with threat-intelligence-driven disclosure (D4). What the record cannot yet support is the stronger vernacular claim of an ongoing arms race; that requires adversary-side measurement this corpus lacks (Sec. 11).

9. Evaluation Validity

Our third insight treats validity as a measurable property of evaluations, not a limitations paragraph. We organize the evidence as a progression: information leakage, then dataset-level invalidity, then oracle manipulation, because each tier changes what a reported number can mean.

9.1. V2/V3: Information Leakage, Measured by Intervention

The cleanest datum in our corpus is controlled. Fang et al. measure GPT-4-based agents on 15 real one-day vulnerabilities under two information conditions: with the CVE description supplied, the agent autonomously exploited 87%; without it, success fell to 7%, while every baseline (GPT-3.5, eight open-source models, ZAP, Metasploit) scored zero in both conditions [6]. This more-than-an-order-of-magnitude difference is produced by a single supplied-knowledge variable, the CVE text acts as a partial answer key, so any capability claim inherits its information condition. We therefore report such results only with their conditions attached, and the same discipline applies to temporal leakage: the benchmark authors report that a substantial subset of vulnerabilities post-dated the relevant model knowledge cutoff, yet supplied CVE descriptions still dramatically altered performance.

9.2. V1/V5: Dataset-Level Invalidity, Measured by Reconstruction

PrimeVul demonstrates that the detection literature's benchmark layer was structurally unsound: label noise, near-duplicate pairs across splits, and pre-split public exposure inflated reported performance to the point where paired, chronologically split evaluation collapses strong-model accuracy toward chance [11]. Risse and Böhme reach a compatible verdict from measurement theory, arguing that popular vulnerability datasets score models on the wrong exam [12]. The corpus also contains a live saturation illustration: Argusee reports 100% on CyberSecEval 2 buffer-overflow cases while its own documentation requires human-supplied entry points [32]. Ceiling scores coexisting with bounded autonomy. Agentic CTF suites inherit a further V1 exposure: their tasks derive from publicly written-up competitions, and no included work quantifies the inflation this induces; we return to it as roadmap experiment E2.

9.3. V4/V5: Oracle Manipulation and Environment Brittleness, Documented Operationally

AIxCC supplied operational evidence on outcome definitions. PoV re-execution establishes behavior only for the tested proof; teams documented mitigation-gaming patches, specifically recompiling with MTE/PAC or wrapping entry points in broad `catch` blocks, that pass PoVs without semantic repair, prompting optional functional tests as secondary oracles and an accuracy modifier that made patch correctness the binding score term [41]. Environment leakage is exemplified by the winning CRS nearly disabling itself when organizers prefixed challenge directories

with "ossfuzz," breaking a "fuzz"-substring heuristic, load-bearing benchmark-specific logic invisible until the environment shifted [41]. On the production side, leaderboard-style counting conflates heterogeneous outcomes: of XBOW's $\sim$1,060 submissions, roughly 39% resolved into duplicate or informative categories rather than new/resolved findings, categories that reflect discovery overlap and program policy as much as agent error, making rank non-decomposable without category-aware accounting [2].

9.4. Administrative Record Correction: A Separate Category

DARPA's closeout corrects its own synthetic-vulnerability count from 70 to 63 with an editor's note [3], and prize accounting requires three scoped figures rather than one ($29.5M cumulative commitment announced in 2023 [62], $8.5M final pool, $30.5M distributed [38], [42]); these figures reflect different accounting scopes. These are administrative record corrections, not measurement-validity failures, but they carry a methodological moral we adopt as policy: even authoritative primary sources require version- and date-aware verification, and press intermediaries inherit whatever instability the primary contains.

9.5. A Contamination-Control Protocol

As a concrete deliverable, we specify the minimal protocol any agentic capability evaluation should publish: (i) *temporal firewall*: freeze the task set at date T, evaluate only models whose training cutoffs precede $T - \Delta$ for disclosed Δ, and release the task set only after evaluation; (ii) *provenance screening*: machine-check every task against public write-up corpora and NVD text up to T, reporting overlap rates rather than asserting cleanliness; (iii) *information-condition disclosure*: state exactly which artifacts (descriptions, patches, stack traces, credentials) are supplied to the agent, Fang-style; (iv) *category-aware outcomes*: report validated/exploitable/duplicate/informative as separate counts with adjudication rules fixed before running; and (v) *adjudication audit*: a second scorer re-labels a random 10% sample with agreement statistics. No included evaluation satisfies all five today; E3 (Sec. 11) operationalizes the gap. On the false-positive front, Xiong and Zhang [63] demonstrate that LLM-based filtering can reduce initial false-positive rates from over 92% to as low as 6.3% in the best configuration, though aggressive filtering can itself suppress true vulnerabilities, suggesting that the contamination-control protocol's adjudication step (v) could be augmented with LLM-assisted triage, provided the false-negative trade-off is explicitly measured.

9.6. The Measured Core, Quantified

Three numbers anchor the progression above; each survives re-examination better than the headline claims it corrects. Fang et al.'s intervention separates information supply from capability: Pass@5 of 86.7% and an overall exploitation rate of 40% for GPT-4 under description-supplied conditions collapse to 7% overall without descriptions, while GPT-3.5, eight open-source models, ZAP, and Metasploit score zero throughout [6], demonstrating that the supplied information condition materially determines performance in this evaluation. PrimeVul's reconstruction moves a state-of-the-art 7B detector from 68.26% F1 on BigVul to 3.09% on its paired, chronologically split evaluation, with GPT-3.5- and GPT-4-class prompting "akin to random guessing" under the most stringent settings [11], showing that dataset construction and evaluation methodology can become a binding constraint on apparent model performance. And XBOW's funnel makes outcome-definition noise concrete at production scale: of $\sim$1,060 submissions, 208 duplicates and 209 informative reports, roughly 39%, resolve into neither new nor fixed findings, so leaderboard position aggregates discovery quality with program-policy artifacts unless category-aware accounting is applied [2]. Three further findings deepen the validity picture. Tang et al. [64] demonstrate that triple-path context augmentation improves LLM-based vulnerability detection, suggesting that input engineering, not just model selection, materially affects reported performance. Camarato et al. [65] show that prompt sensitivity accounts for significant variance in LLM-based vulnerability detection, raising questions about reproducibility when prompts are not standardized. And Huang et al. [66] investigate whether fine-tuned LLMs actually understand vulnerability semantics or merely pattern-match surface features, finding evidence for the latter, a result that qualifies this paper's assumption that fine-tuning transfers genuine security understanding. Complementary mechanistic analysis by Sandoval et al. [67] reveals a Format-Reliability Gap: the same models that generate insecure code correctly identify and explain the vulnerability when asked, with security representations remaining computationally inert until the final layer where format-compliance demands compete with them, suggesting that insecure code generation is an interpretability problem, not a training deficit. Shahriar et al. [68] further show that cognitive heuristics—halo, framing, and anchoring—systematically affect LLM vulnerability detection, with models disproportionately flagging vulnerabilities in code they have seen frequently in training data while missing novel patterns. Each datum predates or operates independently of the others; their convergence on the same methodological conclusion is why we treat validity as measured rather than alleged. The real-world gap is now quantified by independent work: Neef et al. [69] report LLM detection rates of 35–63% on real-world WordPress plugin vulnerabilities, with substantial variance across models, while Dahiya et al. [70] report that Gemini 3.1 Pro reaches 81.3% recall on white-box function-level vulnerability classification, while black-box ground-truth coverage remains only 4–8% without methodology guidance, illustrating how benchmark performance can overstate real-world capability when methodology and tooling scaffolds are absent.

9.7. What Survives

Validity constraints do not render the corpus uninterpretable. Condition-controlled results survive (the 7% no-description condition), third-party adjudication survives (Ada Logics' reproduction of 27 AIxCC-derived issues [38]), and platform triage provides weak external checks on vendor claims. What does not survive is unqualified headline numbers, which are precisely the ones that travel furthest.

10. Threats to Validity

Publication and visibility bias. Our industry stream consists of organizations that *publish*: Google, Anthropic, XBOW, DARKNAVY, OpenSSF. Systems that operate silently (commercial pentesting vendors, state programs, internal red teams) are structurally invisible, so the bounded A3 finding is a statement about documented evidence, not about private capability. Proprietary disclosure is incomplete even where marketing is not.

Survivor and selection bias. Vendors report successes; failed autonomous deployments rarely generate blog posts. XBOW's published funnel (208 duplicates, 209 informative) illustrates the problem, while the corpus does not expose negative results that may have remained unpublished elsewhere.

Source-quality asymmetry. Academic records generally provide methods sections; some vendor records are limited to dashboard-style reporting. We mitigate with quote-level extraction, cross-source corroboration where possible (AIxCC cost tables agree across participant and systematization sources), and explicit source-provenance wording, but asymmetry remains, and one unresolved, vendor-reported incident (GTG-1002) remains a boundary-case limitation.

Rapid model churn. Capabilities tracked here span Gemini 1.5 Pro through mid-2026 models; any conclusion phrased about "LLMs" generally is hostage to a moving baseline. We therefore phrase findings about *records and their conditions*, not about model classes.

Incomplete benchmark access. Publisher APIs (IEEE, ACM) were unavailable during corpus construction; coverage leans on arXiv, DBLP, official indexes, and citation chaining. The field's center of gravity publishes on those channels, but systematic gaps cannot be excluded, and screening operated on first-page result sets per query (documented in Sec. 3).

Subjective classification. Autonomy assignment under the decision-authority test requires judgment at boundaries. We bound it with two rules: classify by documented behavior and by the weakest missing condition, and we publish the record-level assignment fields in the artifact CSV; an independent inter-rater study was not conducted, which we flag as a limitation.

Temporal ambiguity. Several artifacts describe systems whose capability changed between event and publication (XBOW's leaderboard state; Big Sleep's model version). We date claims to their sources' datelines and mark snapshot statistics as such.

Undisclosed human assistance. Platform policies mandate review only where platforms exist; nothing prevents undisclosed human effort behind vendor claims of full automation. Our classifications therefore track *documented* autonomy floors, not ceilings.

Single-corpus induction. All quantitative distributions (14 autonomy-assessed records, including 13 classified and one unresolved; per-cell counts) derive from one team's inclusion decisions under one protocol; the saturation test bounds but does not eliminate corpus-dependence. Where a claim would not survive a different reasonable inclusion rule, we say so in place.

Benchmark contamination. Recent work demonstrates that LLM training data often includes benchmark tasks and their solutions [11], [35]. Our validity taxonomy (V1–V5) names this threat explicitly, and we adopt the five-part contamination-control protocol (Sec. 9) as a concrete mitigation recommendation. However, we cannot guarantee that any included benchmark result is contamination-free; this limits the strength of capability claims derived from benchmark pass rates.

Absence of adversarial evaluation. Our corpus contains no red-team evaluation of the systems we classify. We cannot assess whether the A2 systems we describe would survive adversarial probing designed to find failure modes, edge cases, or exploitation paths that standard benchmarks do not cover. This is a structural limitation of working from published artifacts rather than conducting independent evaluation.

Ethics. This study is documentary: we analyzed published artifacts and did not execute exploits, contact vendors, or handle vulnerability data beyond what primary sources already disclose. Where our corpus touches responsibly disclosed vulnerabilities, identifiers are handled per venue policy, and no new attack capability is introduced beyond quoted primary material.

11. Open Problems: An Experimental Roadmap

Every gap below is stated with a falsifier: the observation that would disconfirm the expected result. We prioritize experiments by how efficiently they resolve load-bearing uncertainties identified in Secs. 5–9.

E1: Isolating autonomy from model scale (I1's unresolved half). Gap: no included work ablates autonomy-level contribution from base-model improvement. Setup: fix a task suite and environment; evaluate the same base model across A1-style fixed pipelines and A2-style dynamic agents, repeated across successive model releases. Controls: identical tools, prompt budgets, and scoring; release-pinned checkpoints. Metric: marginal success attributable to decision authority vs. model generation. Falsifier: if fixed pipelines close the gap to agents as models improve, "autonomy" is re-branding of scale, weakening I1's substantive reading.

E2: Quantifying write-up exposure in agentic CTF suites (V1). Gap: public-solution contamination is widely hypothesized but insufficiently quantified for multi-step agents, and long-horizon security tasks, multi-step discovery chains requiring sustained context, remain highly sensitive to model, project, and configuration: in the May 2026 version, Lee et al. [71] evaluate 183 validated vulnerabilities across V8 and SpiderMonkey, with the strongest frontier configuration reaching 32.0% on V8 and 38.8% on SpiderMonkey, while the open-weight Kimi-K2.6 baseline reaches 11.7% on V8. The two-agent union reaches 37.9% on V8 and 48.8% on SpiderMonkey. Setup: time-split evaluation on challenges disclosed after each model's cutoff, against a matched pre-cutoff set. Controls: difficulty matching via human solve rates; retrieval audits. Metric: within-model success delta across the disclosure boundary. Falsifier: near-zero delta implies CTF benchmarks measure general problem solving, not leaked knowledge, and current headline numbers stand.

E3: Contamination-controlled agentic benchmark (§9 protocol instantiated). Gap: no included evaluation satisfies all five protocol requirements. Setup: build a benchmark under the five-part protocol (temporal firewall, provenance screening, information-condition disclosure, category-aware outcomes, adjudication audit). Controls: release pipeline auditable from task creation. Metric: fraction of prior-suite rankings preserved. Falsifier: high rank correlation with contaminated predecessors would show contamination is not distorting conclusions, bounding I3.

E4: Exploit-aware patch validation. Gap: PoV-passing patches may be semantic non-fixes (mitigation gaming). Setup: differential testing of AIxCC-derived patches under alternate exploit inputs, sanitizer diversity, and semantic-equivalence checks. Controls: human-expert ground truth on a sample. Metric: semantic-repair rate vs. PoV-pass rate. Falsifier: if the rates coincide, PoV oracles suffice and the gaming taxonomy is a curiosity.

E5: The patch/retest loop at ecosystem scale (B4). Gap: B4 exists only as foundation-documented transition; regression impact unmeasured. Setup: instrument CRS-vs-maintainer workflows on live OSS projects (the OpenSSF-hosted tools provide the fleet); track maintainer latency, acceptance, regressions. Controls: pre-registration; maintainers blinded to tool origin. Metric: time-to-fix and regression rate deltas. Falsifier: null effect would show competition-derived tooling did not alter ecosystem patch dynamics, trimming D3's interpretation.

E6: Measuring adversary-side response. Gap: co-evolution evidence is defender-side; attacker responses are anecdotal. Setup: longitudinal measurement of attacker tooling/behavioral markers in incident telemetry shared under defense-pact agreements. Controls: differential exposure to defender publications. Metric: adoption lag of agentic techniques post-disclosure. Falsifier: absence of lag structure supports shared-cause over response-coupling, capping I2 at its current grade.

E7: Decision-impact audit (upgrading I3). Gap: "binding constraint" lacks a documented case of a decision changing on validity correction. Setup: retrospective audit of rankings, funding calls, and procurement that consumed pre-PrimeVul-era results; interview plus document tracing. Metric: count of decisions reversed or re-scored after validity correction. Falsifier: zero documented cases would cap I3 at "important constraint" and justify removing "binding."

12. Conclusion

This systematization asked whether LLM-driven vulnerability work between January 2024 and June 2026 is best understood through model capability or through something else. Our answer, built on a 36-record multi-stream corpus of which 14 records underwent autonomy assessment (13 definitive classifications, one unresolved), is that the field's evaluations, incentives, and discourse now measure progress in *agentic autonomy*, while whether underlying capability moved for reasons other than scale remains an open question.

On I1 (established in framing; unresolved in substance). The autonomy gradient A0–A3 organizes the corpus cleanly: capability probes (A0) persist only as benchmark subjects; pipeline automation (A1) runs at ecosystem scale with humans at review boundaries; task agents (A2) appear across seeded production research, entry-point-assisted auditing, platform-operated pentesting, and competition CRSs integrating discovery, proof, and patching. What the record does not contain, under our four-verb, environment-E2 operational definition, is a single A3 autonomous production hunter. No included evidence met our operational definition of A3. The nearest candidate is an offensive incident, vendor-reported with hallucination caveats and human-selected targets.

On I2 (documented edges; causality bounded). Offense and defense are coupled where the record says so: Project Zero named AIxCC's Team Atlanta as the inspiration for its SQLite variant-analysis campaign; a maintainer extended an AI-found bug's fix to sibling code paths; AIxCC generated documented institutional follow-through: DARPA created transition incentives of up to $200K per finalist team for transition activities, while the competition itself carried prize commitments reported as $29.5M in DARPA's 2023 announcement and $30.5M in OpenSSF's later retrospective; separately, OpenSSF established governance and hosting structures for post-competition CRS development; and threat intelligence seeded a pre-exploitation disclosure. These documented edges do not license the arms-race vernacular: shared-cause explanations survive globally, and attacker-side response remains unmeasured.

On I3 (observed; binding claim pending E7). Validity failures here are not rhetorical. Information conditions change reported exploitation rates by more than an order of magnitude; dataset reconstruction collapses detection SOTA; oracle gaming has a named taxonomy; environment-specific heuristics can materially affect system outcomes; and leaderboard counting conflates outcome categories. Because these measures allocate prizes and rank vendors directly and inform funding decisions through transition programs, evaluation validity is a binding methodological constraint on what this field can claim to know, pending the decision-impact audit (E7) that would confirm this.

What we ask of the next iteration of this literature. Publish information conditions with every capability number; adopt category-aware outcome accounting; evaluate patching against exploit-aware oracles; measure B4 rather than narrate it; and treat autonomy claims by the weakest missing condition, not the strongest available adjective. The tools described in this paper will be judged, fairly or not, by whether they make those practices easier.

Appendix A.
Included Records: Classification, Quality, and Provenance

Complete per-record audit artifact: classification (axes per Sec. IV), quality rubric (reproducibility/baseline-rigor/artifact availability, 0–3), evidence class, and access provenance. Record-level assignment fields ship in `evidence/autonomy-loop-assignments.csv`.

TABLE 4. INCLUDED RECORDS (PART 1/2): IN-WINDOW, BACKGROUND ANCHORS, AND LOOP-TRANSITION. A=AUTONOMY LEVEL; B=LOOP-POSITION SET; R/B/AR = RUBRIC SCORES; ACC = ACCESS STATUS.

Record	Year	Unit	A	B-set / role	Evidence class	R/B/Ar	Acc
In-window autonomy-bearing records (14)							
agentless2024	2024	system(experiment-benchmark)	A1	B3	experiment	3/3/3	PDF
argusee2025	2025	system	A2	B1	production discovery	0/2/0	HTML
artemis2025	2025	experiment	A2	B1+B2	live enterprise-network experiment	2/3/2	PDF
autocoderoever-2024	2024	system(experiment-benchmark)	A2	B3	experiment	3/3/3	PDF
bigsleep2024	2024	system	A2	B1	production discovery	0/2/0	HTML
fang2024one-day	2024	experiment	A2	B2	controlled experiment	2/3/3	PDF
gtg1002-2025	2025	incident-documentation	unresolved	B1+B2	incident report	0/0/0	HTML
ossfuzz-aifixing-2024	2024	experiment	A2	B3	experiment	2/2/2	PDF
ossfuzz-levelingup-2024	2024	system(pipeline)	A1	B1	industrial-system-report	1/1/2	HTML
sweagent2024	2024	system(experiment-benchmark)	A2	B3	benchmark/experiment	3/3/3	PDF
teamatlanta-afc2025	2025	system	A2	B1+B2+B3	competition-participant	1/2/2	HTML
teamatlanta-sqlite2024	2024	system	A2	B1+B2+B3	competition-participant	1/2/2	HTML
tob-buttercup2025	2025	system	A2	B1+B2+B3	competition-participant	1/3/2	HTML
xbow2025	2025	system	A2	B1+B2	production stats	0/1/0	HTML
Background anchors (excluded from primary counts)							
csev12023bg	2023	benchmark	A0	B1	benchmark	2/1/2	PDF
intercode2023bg	2023	benchmark	A1	B2	benchmark	3/2/3	PDF
Loop-transition documentation (B4)							
bloggoogle-summer2025	2025	program-record	n/a	B4 — documents B1 discovery → announced B4 redeployment of Big Sleep to OSS projects	program-announcement	–/–/–	HTML
openssfretro2026	2026	program-record	n/a	B4 — documents post-competition B3→B4 institutionalization (SIG hosting, transition funding, ecosystem stats)	foundation closeout	1/1/2	HTML

TABLE 5. INCLUDED RECORDS (PART 2/2): OFF-AXIS AND CONTEXT RECORDS. A=AUTONOMY LEVEL; B=LOOP-POSITION SET; R/B/AR = RUBRIC SCORES; ACC = ACCESS STATUS.

Record	Year	Unit	A	B-set / role	Evidence class	R/B/Ar	Acc
Off-axis / context records							
csev22024	2024	benchmark	n/a	n/a — probed B1,B2	benchmark	2/2/2	PDF
csev32024	2024	benchmark	n/a	n/a — probed B1,B2	benchmark	2/2/2	PDF
cybench2024	2024	benchmark	n/a	n/a — probed B2	benchmark	3/3/3	PDF
mcpeco2025	2025	measurement-study	n/a	n/a	measurement	2/2/3	PDF
mcpgov2025	2025	documentary-source	n/a	n/a	positional/measurement	1/1/2	PDF
mcppoison2026	2026	documentary-source	n/a	n/a	analysis/experiment	2/2/2	PDF
mcpspec2026	2026	documentary-source	n/a	n/a	analysis	2/2/2	PDF
nvdcve20256965	2025	registry	n/a	n/a	registry	3/3/3	HTML
nyuctfbench2024	2024	benchmark	n/a	n/a — probed B2	benchmark	3/2/3	PDF
primevul2024	2024	dataset	n/a	n/a — B1 detection (dataset standard)	dataset/validity study	3/3/3	PDF
r1slr2024	2024	survey	n/a	n/a	survey	1/1/0	PDF
r2slr2024	2024	survey	n/a	n/a	survey	1/1/0	PDF
r3survey2026	2026	survey	n/a	n/a	survey	1/2/0	PDF
r4csur2025	2025	survey	n/a	n/a	survey	0/0/0	payw.
r5usenix26agentic	2026	survey	n/a	n/a	survey	0/0/0	PDF
r6tosem2026	2026	survey	n/a	n/a	survey	0/0/3	payw.
r7sokaixcc2026	2026	survey(systematization)	n/a	n/a	competition	2/3/2	PDF
rissebohme-wrongexam2024	2024	critique	n/a	n/a	validity-critique	–/–/–	PDF

Appendix B.
Search Log Summary

Stream A queries (arXiv UI, window 2024-01-01..2026-06-30, submitted-date filter): Q1 "large language model" AND "vulnerability detection" (253); Q2 "language model" AND "vulnerability discovery" (42); Q3 "language model" AND "penetration testing" (67); Q4 "language model" AND "CTF" (36); Q5 "LLM agent" AND "exploit" (192); Q6 "large language model" AND "program repair" (202); Q7 "cyber reasoning system" (9); Q8 "AI agent" AND "vulnerability" (172); subtotal 973 raw hits; deduplicated first-page screening union 316 distinct IDs. DBLP API: D1 "large language model" vulnerability (174); D2 "large language model" "penetration testing" (11); D3 LLM CTF benchmark (5); D4 vulnerability repair LLM (24); subtotal 214. Streams B/C: targeted retrieval with per-source verification entries in the artifact manifest. Executed 2026-08-26; full strings and failure notes in repository file `search-log.md`.

Appendix C.
AIxCC and Industrial Timeline

TABLE 6. DATED EVENT CHAIN (PRIMARY SOURCES WHERE AVAILABLE; SECONDARY SYNTHESIS EXPLICITLY IDENTIFIED; GRADES PER SEC. 8).

Date	Event (source)
2023-08	AIxCC announced; ∼$29.5M cumulative program commitment [62]
2024-08	ASC: 42 teams → 7 finalists ($2M each); Atlantis autonomously patches real SQLite FTS5 bug (unscored); official fix e9b919d5 follows [40]
2024-11	Big Sleep reports SQLite stack-buffer-underflow zero-day; cites AIxCC inspiration (edge D1); AFL 150 CPU-h fails to rediscover; no CVE assigned [1]
2024-11	OSS-Fuzz reports 26 vulns from AI-generated fuzz targets incl. CVE-2024-9143 (∼20 years latent) [31]
2025-05	DARKNAVY Argusee finds CVE-2025-37891 (Linux USB); root LPE by human team [32]
2025-06	XBOW #1 US HackerOne; ∼1060 submissions; mandated human review disclosed [2]
2025-07	CVE-2025-6965 threat-intel-seeded disclosure; "first time an AI agent has been used to directly foil" claim [59], [60]
2025-08	AFC finals: 48 challenges/23 repositories in the Trail of Bits account; 18 real vulns found; Atlanta $4M / ToB $3M / Theori $1.5M; $200K/team transition funding [3], [39], [41], [42]
2025-09 (detected), 2025-11 (reported)	Anthropic reports GTG-1002 campaign with an 80–90% execution estimate; the incident remains a single-source, unresolved autonomy assessment [5], [61]
2026-02	R7 AIxCC SoK preprint first released; later August 2026 publication is outside the evidence cutoff; OpenSSF legacy retrospective: CRS SIG, FuzzingBrain 62v/26proj, OSS-CRS 25v/16proj [4], [38]

Appendix D.
A3 Sensitivity Analysis

Table 7 reports counterfactual qualifying-system counts under explicitly relaxed predicates. These scenarios do not change the paper's strict operational definition or its bounded corpus finding.

TABLE 7. COUNTERFACTUAL SENSITIVITY ANALYSIS. LET P DENOTE INDEPENDENT PLANNING/TARGET SELECTION, X EXECUTION, V VALIDATION, R REPORTING, AND E AN E2 TARGET ENVIRONMENT (SEC. 4); THE STRICT PREDICATE IS $P \wedge X \wedge V \wedge R \wedge E$. S1–S3 RELAX EXACTLY R, P, OR E, RESPECTIVELY; S4 RELAXES P, R, AND E TOGETHER. X AND V REMAIN MANDATORY IN EVERY SCENARIO. COUNTS ARE COUNTERFACTUAL QUALIFIERS, NOT REASSIGNMENTS OF THE STRICT TAXONOMY.

Scenario	Condition relaxed	Qualifiers	Closest candidate	Remaining blocker(s)
S0	None (baseline)	0	XBOW	XBOW: R; AIxCC: P, E; ARTEMIS: P, V; GTG-1002: uncertain
S1	R	1	XBOW	AIxCC: P, E; ARTEMIS: P, V; GTG-1002: uncertain
S2	P	0	AIxCC CRS	AIxCC: E; XBOW: R; ARTEMIS: V, R
S3	E	0	AIxCC CRS	AIxCC: P; XBOW: R; ARTEMIS: P, V
S4	P, R, E	2	XBOW; AIxCC CRS	ARTEMIS: V; GTG-1002: uncertain

References

[1] The Big Sleep Team (Google Project Zero and Google DeepMind), "From Naptime to Big Sleep: Using Large Language Models To Catch Vulnerabilities In Real-World Code," Project Zero blog, 11 2024. [Online]. Available: https://projectzero.google/2024/10/from-naptime-to-big-sleep.html

[2] N. Waisman and XBOW, "The Road to Top 1: How XBOW Did It," XBOW company blog, 6 2025. [Online]. Available: https://xbow.com/blog/top-1-how-xbow-did-it

[3] DARPA, "AI Cyber Challenge Marks Pivotal Inflection Point for Cyber Defense," DARPA news release, 2025. [Online]. Available: https://www.darpa.mil/news/2025/aixcc-results

[4] C. Zhang, Y. Park, F. Fleischer, Y.-F. Fu, J. Kim, D. Kim, Y. Kim, Q. Xu, A. Chin, Z. Sheng, H. Zhao, M. Pelican, D. J. Musliner, J. Huang, J. Silliman, M. McDaniel, J. Casavant, I. Goldthwaite, N. Vidovich, M. Lehman, and T. Kim, "SoK: DARPA's AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned," in *USENIX Security Symposium*, 2026, arXiv preprint Feb 2026 v1, v5 Aug 2026; to appear USENIX Security 2026. [Online]. Available: https://arxiv.org/abs/2602.07666

[5] Anthropic Threat Intelligence, "Disrupting the first reported AI-orchestrated cyber espionage campaign," Anthropic newsroom, 11 2025. [Online]. Available: https://www.anthropic.com/news/disrupting-AI-espionage

[6] R. Fang, R. Bindu, A. Gupta, and D. Kang, "LLM Agents can Autonomously Exploit One-day Vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2404.08144

[7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based Software Engineering Agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[8] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, R. Prakash, D. Kapil, Y. Kozyrakis, D. LeBlanc, J. Milazzo, A. Straumann, G. Synnaeve, V. Vontimitta, S. Frolov, S. Whitman, and J. Saxe, "Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2312.04724

[9] M. Bhatt, S. Chennabasappa, S. Wan, Y. Li, C. Nikolaidis, D. Song, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil, D. Molnar, S. Whitman, and J. Saxe, "CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models," 2024. [Online]. Available: https://arxiv.org/abs/2404.13161

[10] S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, Y. Li, S. Whitman, S. Ding, V. Ionescu, and J. Saxe, "CYBERSECEVAL 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models," 2024. [Online]. Available: https://arxiv.org/abs/2408.01605

[11] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?" 2024. [Online]. Available: https://arxiv.org/abs/2403.18624

[12] N. Risse, J. Liu, and M. Böhme, "Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection," 2024. [Online]. Available: https://arxiv.org/abs/2408.12986

[13] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks," 2024, arXiv preprint Dec 2023, v3 Jul 2024. [Online]. Available: https://arxiv.org/abs/2312.12575

[14] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," 2024, arXiv May 2024; key year 2025 is alias. [Online]. Available: https://arxiv.org/abs/2405.17238

[15] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao, "InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback," 2023. [Online]. Available: https://arxiv.org/abs/2306.14898

[16] M. Shao, S. Jancheska, M. Udeshi, B. Dolan-Gavitt, H. Xi, K. Milner, B. Chen, M. Yin, S. Garg, P. Krishnamurthy, F. Khorrami, R. Karri, and M. Shafique, "NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security," 2024. [Online]. Available: https://arxiv.org/abs/2406.05590

[17] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, C. Menders, J. W. Lin, E. Jones, G. Hussein, S. Liu, D. Jasper, P. Peetathawatchai, A. Glenn, V. Sivashankar, D. Zamoshchin, L. Glikbarg, D. Askaryar, M. Yang, T. Zhang, R. Alluri, N. Tran, R. Sangpisit, P. Yiorkadjis, K. Osele, G. Raghupathi, D. Boneh, D. E. Ho, and P. Liang, "Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv preprint Aug 2024; to appear ICLR 2025. [Online]. Available: https://arxiv.org/abs/2408.08926

[18] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 847–864. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[19] X. Li and X. Gao, "A First Look at the Security Issues in the Model Context Protocol Ecosystem," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2026, arXiv preprint Oct 2025; to appear DSN 2026. [Online]. Available: https://arxiv.org/abs/2510.16558

[20] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard, "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning," 2026. [Online]. Available: https://arxiv.org/abs/2603.22489

[21] N. Maloyan and D. Namiot, "Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities," 2026. [Online]. Available: https://arxiv.org/abs/2601.17549

[22] H. Errico, J. Ngiam, and S. Sojan, "Securing the Model Context Protocol (MCP): Risks, Controls, and Governance," 2025. [Online]. Available: https://arxiv.org/abs/2511.20920

[23] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When LLMs Meet Cybersecurity: A Systematic Literature Review," 2024. [Online]. Available: https://arxiv.org/abs/2405.03644

[24] E. Basic and A. Giaretta, "From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security," 2024. [Online]. Available: https://arxiv.org/abs/2412.15004

[25] Z. He, J. Dong, Z. Li, T. Chen, G. Deng, F. Luo, J. Ji, Y. Cao, and X. Luo, "A Survey of LLM-Driven Penetration Testing: Taxonomy, Co-Evolution, and Open Challenges," 2026. [Online]. Available: https://arxiv.org/abs/2607.02605

[26] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights," *ACM Computing Surveys*, vol. 58, no. 5, pp. 134:1–134:35, 2025. [Online]. Available: https://doi.org/10.1145/3769082

[27] J. Kim, W. Guo, and D. Song, "SoK: Attack and Defense Landscape of Agentic AI Systems," in *USENIX Security Symposium*, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/kim-juhee-agentic

[28] S. Kaniewski, F. Schmidt, M. Enzweiler, M. Menth, and T. Heer, "A Systematic Literature Review on Detecting Software Vulnerabilities with Large Language Models," *ACM Transactions on Software Engineering and Methodology*, 2026, 263 studies Jan 2020–Nov 2025; preprint Jul 2025. [Online]. Available: https://doi.org/10.1145/3815425

[29] M. Chen, H. Cheng, T. Su, M. Chen, W. Cai, and H. Zou, "Evaluating Large Language Models in Cybersecurity: A Systematic Taxonomy and Empirical Analysis," *Electronics*, vol. 15, no. 10, p. 2222, 2026. [Online]. Available: https://www.mdpi.com/2079-9292/15/10/2222

[30] S. Halder, S. Saxena, K. Kadaba Shrish, and M. Thiyagarajan, "SecLens: Role-Specific Evaluation of LLMs for Security Vulnerability Detection," 2026. [Online]. Available: https://arxiv.org/abs/2604.01637

[31] O. Chang, D. Liu, and J. Metzman, "Leveling Up Fuzzing: Finding More Vulnerabilities with AI," Google Security Blog, 11 2024. [Online]. Available: https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html

[32] DARKNAVY, "Argusee: A Multi-Agent Collaborative Architecture for Automated Vulnerability Discovery," DARKNAVY blog, 5 2025. [Online]. Available: https://www.darknavy.org/blog/argusee_a_multi_agent_collaborative_architecture_for_automated_vulnerability_discovery/

[33] J. W. Lin, E. K. Jones, D. J. Jasper, E. J.-s. Ho, A. Wu, A. T. Yang, N. Perry, A. Zou, M. Fredrikson, J. Z. Kolter, P. Liang, D. Boneh, and D. E. Ho, "Comparing AI Agents to Cybersecurity Professionals in Real-World Penetration Testing," 2025. [Online]. Available: https://arxiv.org/abs/2512.09882

[34] X. Liu, S. Song, Z. Zhang, H. Lan, J. Zeng, M. Wu, M. Heinrich, Y. Sun, and C. Zhang, "Agora: Toward Autonomous Bug Detection in Production-Level Consensus Protocols with LLM Agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.29910

[35] Y. Zhu, A. Kellermann, D. Bowman, P. Li, A. Gupta, A. Danda, R. Fang, C. Jensen, E. Ihli, J. Benn, J. Geronimo, A. Dhir, S. Rao, K. Yu, T. Stone, and D. Kang, "CVE-bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 79 850–79 867. [Online]. Available: https://proceedings.mlr.press/v267/zhu25i.html

[36] P. He, A. Fox, L. Miculicich, S. Friedli, D. Fabian, B. Gokturk, J. Tang, C.-Y. Lee, T. Pfister, and L. T. Le, "Co-RedTeam: Orchestrated Security Discovery and Exploitation with LLM Agents," 2026. [Online]. Available: https://arxiv.org/abs/2602.02164

[37] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, "CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=2YvbLQEdYt

[38] H. Woeste and OpenSSF, "Hack to the Future: The Impact and Legacy of the DARPA AIxCC Challenge," OpenSSF blog, 5 2026. [Online]. Available: https://openssf.org/blog/2026/05/12/hack-to-the-future-the-impact-and-legacy-of-the-darpa-aixcc-challenge/

[39] D. Guido, "Trail of Bits' Buttercup Wins 2nd Place in AIxCC Challenge," Trail of Bits blog, 8 2025, trail of Bits. [Online]. Available: https://blog.trailofbits.com/2025/08/09/trail-of-bits-buttercup-wins-2nd-place-in-aixcc-challenge/

[40] H. Zhao, "Autonomously Uncovering and Fixing a Hidden Vulnerability in SQLite3 with an LLM-Based System," Team Atlanta blog, 8 2024, team Atlanta. [Online]. Available: https://team-atlanta.github.io/blog/post-asc-sqlite/

[41] T. Kim, "AIxCC Final and Team Atlanta," Team Atlanta blog, 8 2025, team Atlanta. [Online]. Available: https://team-atlanta.github.io/blog/post-afc

[42] DARPA, "DARPA's AI Cyber Challenge Releases Scoring Guide for the 8.5 Million Dollar Final Competition," DARPA news release, 2025. [Online]. Available: https://www.darpa.mil/news/2025/ai-cyber-challenge-scoring

[43] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[44] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous Program Improvement," 2024. [Online]. Available: https://arxiv.org/abs/2404.05427

[45] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, D. Liu, A. Arya, O. Chang, and A. Roychoudhury, "Fixing Security Vulnerabilities with AI in OSS-Fuzz," 2024. [Online]. Available: https://arxiv.org/abs/2411.03346

[46] S. Ullah, P. Balasubramanian, W. Guo, A. Burnett, H. Pearce, C. Kruegel, G. Vigna, and G. Stringhini, "From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs," 2025. [Online]. Available: https://arxiv.org/abs/2509.01835

[47] Q. Xu, Z. Sheng, Z. Chen, and J. Huang, "A Systematic Study of LLM-Based Architectures for Automated Patching," 2026. [Online]. Available: https://arxiv.org/abs/2603.01257

[48] Y. Kim, S. Shin, H. Kim, and J. Yoon, "Logs In, Patches Out: Automated Vulnerability Repair via Tree-of-Thought LLM Analysis," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/kim-youngjoon

[49] M. Fakih, R. Dharmaji, H. Bouzidi, G. Q. Araya, O. Ogundare, and M. A. Al Faruque, "LLM4CVE: Enabling Iterative Automated Vulnerability Repair with Large Language Models," 2025. [Online]. Available: https://arxiv.org/abs/2501.03446

[50] Y. Nong, H. Yang, L. Cheng, H. Hu, and H. Cai, "APPATCH: Automated Adaptive Prompting Large Language Models for Real-World Software Vulnerability Patching," in *Proceedings of the 34th USENIX Security Symposium*, 2025, pp. 4481–4500, arXiv preprint Aug 2024. [Online]. Available: https://arxiv.org/abs/2408.13597

[51] Q. Wu, Y. Xiao, D. Kirat, K. Eykholt, J. Jang, and D. L. Schales, "One Bug, Hundreds Behind: LLMs for Large-Scale Bug Discovery," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. PMLR, 2026, arXiv preprint Oct 2025; to appear ICML 2026. [Online]. Available: https://arxiv.org/abs/2510.14036

[52] X. Meng, Z. Ma, P. Gao, and C. Peng, "An Empirical Study on LLM-based Agents for Automated Bug Fixing," 2024, arXiv Nov 2024; key year 2026 is alias. [Online]. Available: https://arxiv.org/abs/2411.10213

[53] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," 2025. [Online]. Available: https://arxiv.org/abs/2501.18160

[54] N. Korda and G. Evron, "OpenAnt: LLM-Powered Vulnerability Discovery Through Code Decomposition, Adversarial Verification, and Dynamic Testing," 2026. [Online]. Available: https://arxiv.org/abs/2606.19149

[55] F. Li, J. Jiang, D. Chen, and Y. Xiong, "LLM-based Vulnerability Detection at Project Scale: An Empirical Study," 2026. [Online]. Available: https://arxiv.org/abs/2601.19239

[56] W. Charoenwet, K. Tantithamthavorn, P. Thongtanunam, H. Y. Lin, M. Jeong, and M. Wu, "AgenticSCR: An Autonomous Agentic Secure Code Review for Immature Vulnerabilities Detection," 2026. [Online]. Available: https://arxiv.org/abs/2601.19138

[57] Z. Wei, J. Zeng, M. Wen, Z. Yu, K. Cheng, Y. Zhu, J. Guo, S. Zhou, L. Yin, X. Su, and Z. Ma, "PATCHEVAL: A New Benchmark for Evaluating LLMs on Patching Real-World Vulnerabilities," 2025. [Online]. Available: https://arxiv.org/abs/2511.11019

[58] P. Przymus, A. Happe, and J. Cito, "Adversarial Bug Reports as a Security Risk in Language Model-Based Automated Program Repair," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. ACM, 2026, arXiv preprint Sep 2025; to appear MSR 2026. [Online]. Available: https://arxiv.org/abs/2509.05372

[59] Walker, Kent, "A Summer of Security: Empowering Cyber Defenders with AI," blog.google, 7 2025. [Online]. Available: https://blog.google/innovation-and-ai/technology/safety-security/cybersecurity-updates-summer-2025/

[60] NVD / Google CNA, "NVD Vulnerability Detail," National Vulnerability Database, 7 2025. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2025-6965

[61] Anthropic Threat Intelligence, "Disrupting the First Reported AI-Orchestrated Cyber Espionage Campaign (Full Report)," Anthropic (PDF), 11 2025. [Online]. Available: https://www-cdn.anthropic.com/d7dd50dd1185f59be051b307150d877f2b82bd2c.pdf

[62] DARPA, "DARPA Launches the AI Cyber Challenge," DARPA news release, 2023. [Online]. Available: https://www.darpa.mil/news/2023/ai-cyber-challenge-opens

[63] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," 2026. [Online]. Available: https://arxiv.org/abs/2601.22952

[64] W. Tang, X. Zhang, J. Liu, J. Xiao, X. Xiao, J. Yang, Y. Ma, Z. Liu, Z. Li, Z. Wang, W. Luo, Q. Li, L. Wang, and P. Xiangli, "VulTriage: Triple-Path Context Augmentation for LLM-Based Vulnerability Detection," 2026. [Online]. Available: https://arxiv.org/abs/2605.09461

[65] S. J. Camarato, Y. Hmaiti, M. Ghadamian, and D. Mohaisen, "PromptAudit: Auditing Prompt Sensitivity in LLM-Based Vulnerability Detection," 2026. [Online]. Available: https://arxiv.org/abs/2605.24171

[66] F. Huang, Y. Sun, F. Zhang, Z. Yang, H. Liu, and Y. Liu, "Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap," 2026. [Online]. Available: https://arxiv.org/abs/2601.22655

[67] G. A. Sandoval, B. Dolan-Gavitt, and S. Garg, "Surgical Repair of Insecure Code Generation in LLMs: From Mechanistic Diagnosis to Deployment-Ready Intervention," 2026. [Online]. Available: https://arxiv.org/abs/2604.16697

[68] A. Shahriar, H. Cai, H. Benkraouda, G. Wang, and Z. B. Celik, "Words Speak Louder Than Code: Investigating Cognitive Heuristics in LLM-Based Code Vulnerability Detection," 2026. [Online]. Available: https://arxiv.org/abs/2606.30587

[69] S. Neef, L. Jungnickel, A. B. Buchholz, V. Spence, and V. B. Gonzalez, "Evaluating LLMs for Real-World Web Vulnerability Detection," 2026. [Online]. Available: https://arxiv.org/abs/2606.21397

[70] V. Dahiya, S. Nehra, V. Dholariya, B. Shangari, and C. Khatri, "Are Frontier LLMs Ready for Cybersecurity? Evidence for Vertical Foundation Models from Dual-Mode Vulnerability Benchmarks," 2026. [Online]. Available: https://arxiv.org/abs/2605.23243

[71] H. Lee, J. Liu, D. Kim, Z. Zhang, C. S. Xia, and L. Zhang, "SEC-bench Pro: Can Language Models Solve Long-Horizon Software Security Tasks?" 2026, arXiv v1, May 26, 2026; 183 validated vulnerabilities. [Online]. Available: https://arxiv.org/abs/2605.26548v1